TECHNICAL PROJECT REPORT

INTERACTIVE LUNAR ECLIPSE SIMULATION USING JAVA APPLET

A graphical and animated demonstration of the Sun–Earth–Moon alignment and Earth's shadow

Project Title	Interactive Lunar Eclipse Simulation Using Java Applet
Programming Language	Java
Graphics Library	Java AWT / Graphics2D
Animation Technique	Multithreading and periodic repainting
Project Category	Computer Graphics / Java Programming
Runtime Target	JDK 8 legacy Applet environment

Submitted by: _____

Department / Class: _____

Institution: _____

Academic Session: _____

Abstract

This project presents an interactive graphical simulation of a lunar eclipse using the Java Applet framework and Abstract Window Toolkit (AWT). The application illustrates the alignment of the Sun, Earth and Moon, and visualizes Earth's shadow as two regions: the umbra (the darker inner shadow) and the penumbra (the lighter outer shadow). The user can start, pause and reset the animation, select a simplified penumbral, partial or total eclipse mode, and adjust animation speed. A dedicated animation thread updates the Moon's screen position while a custom AWT canvas renders the scene using Java2D. The simulation is educational and schematic; it is not a scale-accurate astronomical model or a prediction tool.

1. Introduction

A lunar eclipse occurs when Earth passes between the Sun and the Moon and the Moon enters Earth's shadow. This alignment is possible near a full Moon when the three bodies are sufficiently close to the same orbital plane. Earth's shadow consists of the umbra, where direct sunlight is blocked, and the penumbra, where sunlight is only partially blocked.

The purpose of this project is to convert this astronomical concept into a visual and interactive computer simulation. Animation helps learners observe the progression of the Moon through the shadow regions rather than relying only on a static diagram.

2. Problem Statement

The geometry of a lunar eclipse can be difficult to visualize from a written description or a two-dimensional static illustration. A compact educational application is therefore required to demonstrate the Sun–Earth–Moon arrangement, distinguish the umbra from the penumbra, and show how different eclipse categories relate to the Moon's path through Earth's shadow.

3. Objectives

• Represent the Sun, Earth and Moon on a graphical display.

• Illustrate the umbra and penumbra behind Earth.

• Animate the Moon's movement through the shadow region.

• Demonstrate penumbral, partial and total eclipse modes.

• Provide user controls for starting, pausing, resetting and changing animation speed.

• Apply Java concepts including inheritance, event handling, graphics programming and multithreading.

4. Scientific Background

4.1 Umbra and Penumbra

Umbra: the darkest central part of a shadow, where the direct light source is completely obscured. **Penumbra:** the lighter outer part of a shadow, where the light source is only partially obscured.

4.2 Types of Lunar Eclipse

Penumbral lunar eclipse: the Moon passes through some or all of Earth's penumbra, producing subtle dimming. **Partial lunar eclipse:** part of the Moon enters Earth's umbra. **Total lunar eclipse:** the entire Moon enters Earth's umbra. During totality, the Moon may appear copper-red because Earth's atmosphere refracts and filters sunlight into the shadow.

4.3 Model Simplification

The applet uses a two-dimensional schematic. The Sun, Earth, Moon and shadow dimensions are not drawn to astronomical scale. The selectable modes use simplified screen-coordinate paths and thresholds to demonstrate the concepts. Real eclipse geometry depends on three-dimensional orbital positions, angular diameters, orbital inclination

and time.

5. Software and Hardware Requirements

Category	Requirement
Programming language	Java
API	java.applet.Applet, java.awt, java.awt.event, java.awt.geom, java.awt.image
Development kit	JDK 8 recommended for the legacy Applet API and appletviewer
Editor / IDE	Any Java-compatible editor or IDE
Operating system	Windows, macOS or Linux with a compatible Java runtime
Hardware	A standard desktop/laptop with display and keyboard/mouse

6. System Design

6.1 Architecture

The application follows a simple event-driven architecture. The Applet class manages lifecycle methods and user-interface controls. The animation thread updates the Moon's position. The custom SceneCanvas is responsible for drawing the background, celestial bodies, shadow polygons, labels and legend. The canvas uses an off-screen BufferedImage to reduce visible flicker.

6.2 Main Components

• **LunarEclipseApplet:** main class; extends Applet and implements Runnable and AWT listener interfaces.

• **SceneCanvas:** inner Canvas class; renders the simulation with Graphics2D.

• **Animation thread:** periodically changes the Moon's x-coordinate and requests repainting.

• **Control panel:** AWT Button, Choice, Scrollbar and Label components.

• **Stage classifier:** maps the Moon's relative position to a descriptive eclipse stage.

6.3 Data and Control Flow

```
Applet init()
    |
    v
Create canvas and AWT controls
    |
    v
Applet start() -> create animation thread
    |
    v
User selects eclipse mode / presses Start
    |
    v
Animation loop: update Moon position
    |
    v
Classify stage -> repaint canvas
    |
    v
Draw Sun, Earth, Moon, umbra, penumbra and labels
    |
    +---- Pause / Reset / Change mode / Stop
```

7. Implementation Details

7.1 Applet Lifecycle

init() creates the layout, drawing canvas and controls. **start()** starts the animation thread if it does not already exist. **run()** contains the periodic animation loop. **stop()** stops the loop when the applet is stopped. **destroy()** performs final cleanup.

7.2 Graphics Rendering

The custom Canvas overrides paint(Graphics) and update(Graphics). It creates an off-screen BufferedImage and draws through Graphics2D with anti-aliasing. The Sun is represented by concentric circles and a radial gradient; Earth is represented by a blue circle with simplified land shapes; the Moon is a light circular body. Polygon geometry illustrates the expanding penumbra and narrowing umbra behind Earth.

7.3 Animation and Threading

A separate Thread executes run(). While the animation is active, updateMoonPosition() increments or decrements the Moon's horizontal coordinate according to the selected speed. The thread sleeps briefly between updates, then requests a repaint. The running and paused flags are volatile so that changes made by the AWT event-dispatch thread are visible to the animation thread.

7.4 Event Handling

The class implements ActionListener for Start, Pause and Reset buttons; ItemListener for eclipse-mode selection; and AdjustmentListener for the speed scrollbar. Each event updates the simulation state and requests a redraw where required.

7.5 Eclipse Stage Classification

The application compares the Moon's horizontal coordinate with a reference point near the center of the shadow. In Total mode, the central interval is labelled totality, the adjacent interval partial, and the wider interval penumbral. Partial mode suppresses the totality label; Penumbral mode labels only the penumbral interval. These thresholds are pedagogical approximations, not astronomical calculations.

8. Algorithm

```
1. Initialize the applet and create the drawing canvas.
2. Add Start, Pause, Reset, eclipse-type and speed controls.
3. Set the Moon's initial position and default mode.
4. Start the animation thread.
5. If animation is active:
      a. Update the Moon's x-coordinate using the selected speed.
      b. Reverse direction at the path boundaries.
      c. Determine the current illustrative eclipse stage.
      d. Request canvas repainting.
6. In paint(), draw the star field and title.
7. Draw the Sun and sunlight reference rays.
8. Draw Earth's penumbra and umbra shadow polygons.
9. Draw Earth, the Moon, labels and legend.
10. Repeat until the applet is stopped.
```

9. How to Compile and Execute

The Java Applet API is a legacy technology. Use a JDK 8 installation that includes appletviewer. Modern browsers do not support Java Applets.

Step 1: Open a terminal in the project directory.
Step 2: Compile the source: `javac -source 1.8 -target 1.8 -d . src/LunarEclipseApplet.java`
Step 3: Run the HTML host page: `appletviewer LunarEclipse.html`

If appletviewer is unavailable in the installed JDK, use a JDK 8 distribution that includes the legacy tool. Do not expect the HTML page to run in Chrome, Firefox, Safari or other modern browsers.

10. User Guide

• Select Total, Partial or Penumbral from the Eclipse type list.

• Press Start to animate the Moon along the illustrative path.

• Press Pause to freeze the current frame.

• Press Reset to return the Moon to its starting position.

• Move the speed scrollbar to change the animation step size.

• Observe the stage text and the umbra/penumbra legend.

11. Testing and Results

Test ID	Action	Expected result
T01	Launch applet	Scene, controls and initial Moon position appear
T02	Press Start	Moon begins moving
T03	Press Pause	Moon stops at its current position
T04	Press Reset	Moon returns to initial position
T05	Select Total	Total-mode stage labels are enabled
T06	Select Partial	Partial and penumbral labels are shown; no totality label
T07	Select Penumbral	Only penumbral or outside-shadow labels appear
T08	Change speed	Animation step size changes
T09	Stop applet	Animation thread is interrupted and stops

The test cases define expected behavior. Actual test results should be recorded after running the applet in the target JDK 8 environment. Attach screenshots of the initial state, each eclipse mode, and the controls to the final college submission.

12. Limitations

• The scene is two-dimensional and not drawn to scale.

• The Moon's path and stage thresholds are simplified screen-coordinate logic.

• The application does not calculate real orbital elements or predict eclipse dates.

• The Moon's red coloration is a visual illustration rather than a physical light-scattering simulation.

• Java Applets are deprecated and require a legacy execution environment.

13. Future Scope

• Replace the simplified path with orbital calculations using real astronomical parameters.

• Add a time control and date-based eclipse demonstrations.

• Model the changing apparent size of the umbra and penumbra more accurately.

• Add narration, tooltips and a guided educational mode.

• Port the simulation to Java Swing or JavaFX for modern desktop compatibility, or to a web-based graphics platform if permitted.

14. Conclusion

The Interactive Lunar Eclipse Simulation demonstrates how Java can be used to create a graphical and animated educational application. It combines AWT user-interface components, Graphics2D rendering, event handling and multithreading to communicate the relationship between the Sun, Earth, Moon and Earth's shadow. The project

provides a practical introduction to computer graphics and animation while reinforcing the scientific distinction between penumbral, partial and total lunar eclipses. Its geometry is intentionally schematic, and a precision astronomy model would require more advanced orbital calculations.

15. References

• Oracle, Java Platform Standard Edition 8 API Specification: java.applet.Applet, java.awt.Graphics, java.awt.Graphics2D, java.awt.Canvas, java.awt.event.

• NASA Science, Eclipses: https://science.nasa.gov/eclipses/

• NASA, Lunar Eclipses: https://science.nasa.gov/moon/eclipses/

• Java SE 8 Documentation: https://docs.oracle.com/javase/8/docs/

Appendix A – Source Code

The complete source file is supplied separately as **src/LunarEclipseApplet.java** in the project package. It should be submitted together with this report and the HTML host file.